

OPTIMIZATION ALGORITHMS · MASTER'S COURSE

Local Descent & Optimization Methods

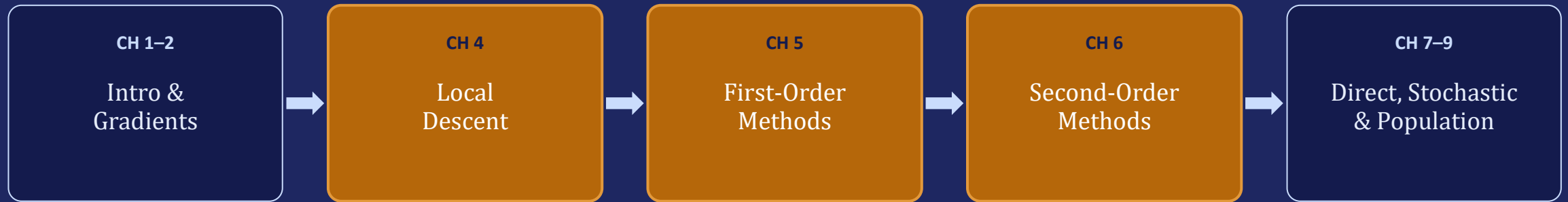
Lecture 2 — From a Gradient to an Algorithm: Descent, First-Order, and Second-Order Methods

- Local descent framework
- First-order methods
- Second-order methods

Based on Kochenderfer & Wheeler, Algorithms for Optimization (MIT Press) — Chapters 4, 5 & 6

Course Roadmap

Lecture 1 built the gradient toolbox. Today's three chapters show how that gradient (or Hessian) actually drives an optimization algorithm.



Today's lecture

Chapters 7–9 (direct, stochastic, and population methods) round out the algorithm landscape from Lecture 1 and are natural next steps.

PART A · CHAPTER 4

Local Descent

The general framework: direction, step size, and when to stop

4.1 DESCENT DIRECTION ITERATION

A General Recipe for Local Descent

Nearly every algorithm in this lecture follows the same iterative pattern:

1

Check termination

Has $\mathbf{x}(k)$ satisfied a stopping condition? If so, stop.

2

Choose a direction

Determine descent direction $\mathbf{d}(k)$ from local information (gradient or Hessian).

3

Choose a step size

Determine step size / learning rate $\alpha(k)$, e.g. via line search.

4

Update

Compute the next design point and repeat.

$$\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} + \alpha^{(k)} \mathbf{d}^{(k)}$$

Different algorithms are just different ways of choosing $\mathbf{d}$ and α . Chapters 5 and 6 are, at heart, catalogs of these choices.

4.2 LINE SEARCH

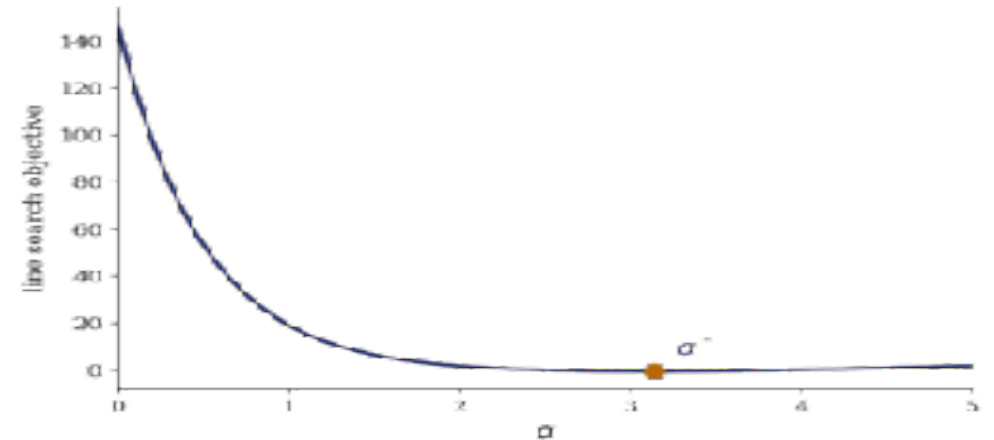
Choosing the Step Size Exactly

Given a descent direction $\mathbf{d}$, line search picks the step factor α that minimizes the objective along that direction — a univariate optimization problem:

$$\text{minimize}_{\alpha} f(\mathbf{x} + \alpha \mathbf{d})$$

Any univariate optimization method from Chapter 3 (e.g. bracketing + golden section, or the Brent-Dekker method) can solve this.

A disadvantage: optimizing α to high precision at every single step is often wasteful. It's common to find a reasonable value quickly and move on — this is approximate line search, next.



4.3 APPROXIMATE LINE SEARCH

Sufficient Decrease (the Armijo Condition)

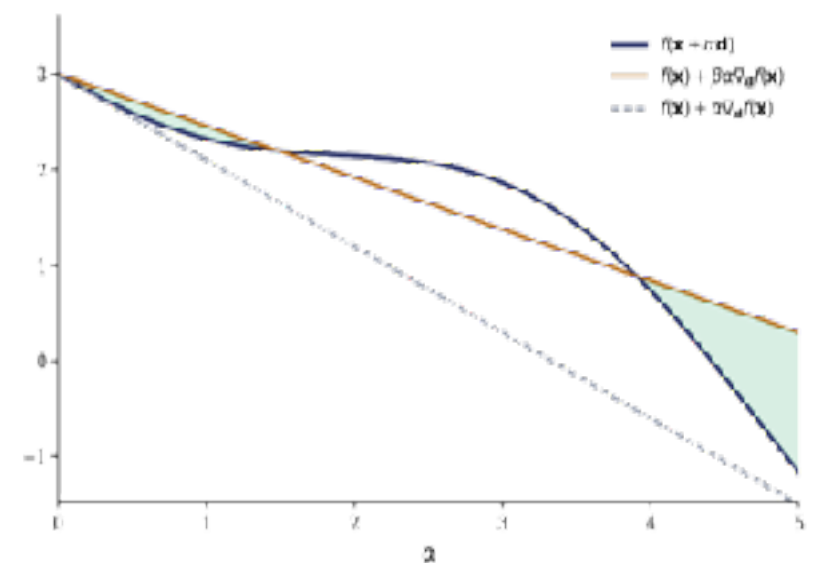
Rather than solving the line search exactly, we just require that α cause a sufficient decrease in f :

$$f(\mathbf{x}^{(k+1)}) \leq f(\mathbf{x}^{(k)}) + \beta\alpha \nabla_{\mathbf{d}^{(k)}} f(\mathbf{x}^{(k)})$$

With $\beta \in [0,1]$ (often 10^{-4}). If $\beta = 0$, any decrease is acceptable; if $\beta = 1$, the decrease must match the first-order prediction.

Backtracking line search: start with a large α and shrink it by a constant factor until the sufficient decrease condition holds. Simple, cheap, and guaranteed to succeed for a valid descent direction.

The shaded region shows step sizes α that satisfy sufficient decrease.



4.3 APPROXIMATE LINE SEARCH

The Wolfe Conditions

Sufficient decrease alone can be satisfied by tiny, overly cautious steps. A curvature condition rules this out by requiring the next slope to be shallower:

$$\nabla_{\mathbf{d}^{(k)}} f(\mathbf{x}^{(k+1)}) \geq \sigma \nabla_{\mathbf{d}^{(k)}} f(\mathbf{x}^{(k)})$$

The strong Wolfe condition tightens this further, also ruling out slopes that are too positive:

$$|\nabla_{\mathbf{d}^{(k)}} f(\mathbf{x}^{(k+1)})| \leq -\sigma \nabla_{\mathbf{d}^{(k)}} f(\mathbf{x}^{(k)})$$

First Wolfe condition

= sufficient decrease (Armijo)

Second Wolfe condition

= curvature condition

4.4 TRUST REGION METHODS

Constraining the Step to a Trusted Region

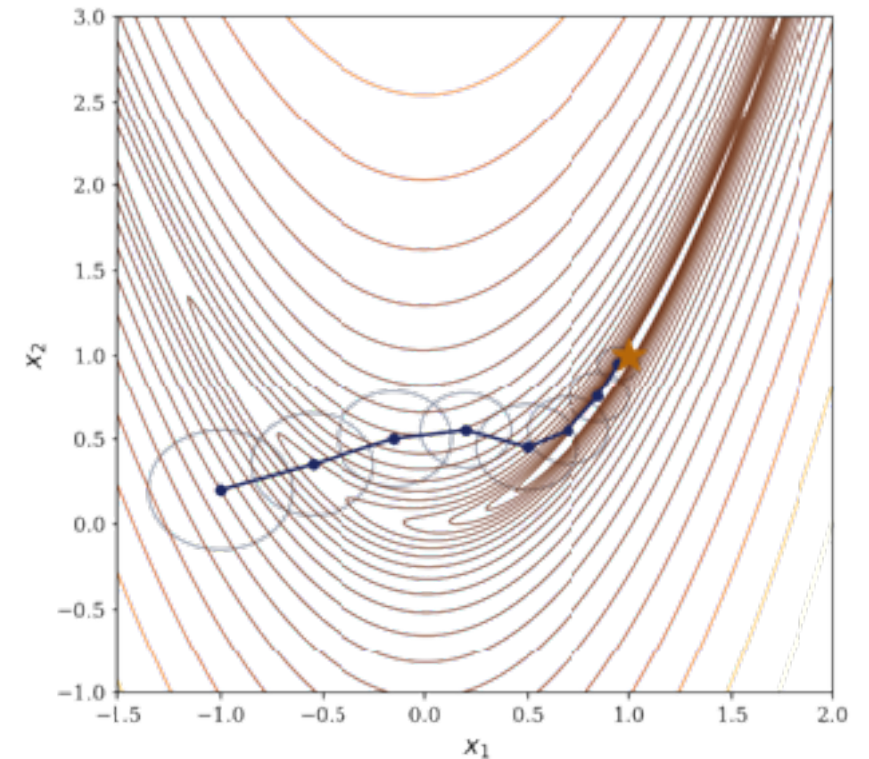
Instead of choosing a direction then a step size, trust region methods first bound the step, then minimize a local model within that bound:

$$\text{minimize}_{\mathbf{x}'} \hat{f}(\mathbf{x}') \quad \text{subject to } \|\mathbf{x} - \mathbf{x}'\| \leq \delta$$

The radius δ is expanded or contracted based on how well the model predicted the actual improvement:

$$\eta = \frac{f(\mathbf{x}) - f(\mathbf{x}')}{f(\mathbf{x}) - \hat{f}(\mathbf{x}')}$$

$\eta \approx 1$: model was accurate $\rightarrow$ expand δ .
 η small: model was poor $\rightarrow$ contract δ .



Trust regions shrink and grow to track how far the local model (e.g. a second-order Taylor expansion) can be trusted on the Rosenbrock function.

4.5 TERMINATION CONDITIONS

When Do We Stop?

Maximum iterations

$$k > k_{\text{max}}$$

Cap the number of steps (or elapsed time).

Absolute improvement

$$f(\mathbf{x}^{(k)}) - f(\mathbf{x}^{(k+1)}) < \epsilon_a$$

Stop when the objective stops improving by more than ϵ_a .

Relative improvement

$$f(\mathbf{x}^{(k)}) - f(\mathbf{x}^{(k+1)}) < \epsilon_r |f(\mathbf{x}^{(k)})|$$

Same idea, scaled by the current objective value.

Gradient magnitude

$$\|\nabla f(\mathbf{x}^{(k+1)})\| < \epsilon_g$$

Stop when the gradient is small — near a stationary point.

In practice, algorithms combine several of these — and may add random restarts when multiple local minima are likely.

Part A — Key Takeaways

- 1 Local descent = repeatedly choose a direction d , choose a step size α , and update x .
- 2 Exact line search solves a univariate optimization; approximate line search (backtracking) is cheaper.
- 3 The Wolfe conditions (sufficient decrease + curvature) keep approximate line search well-behaved.
- 4 Trust region methods bound the step and adapt the trust radius based on model accuracy.
- 5 Termination combines iteration limits, improvement thresholds, and gradient magnitude.

PART B · CHAPTER 5

First-Order Methods

Gradient descent, conjugate gradient, momentum, and adaptive learning rates

5.1 GRADIENT DESCENT

The Direction of Steepest Descent

The most intuitive descent direction is simply the negative (normalized) gradient:

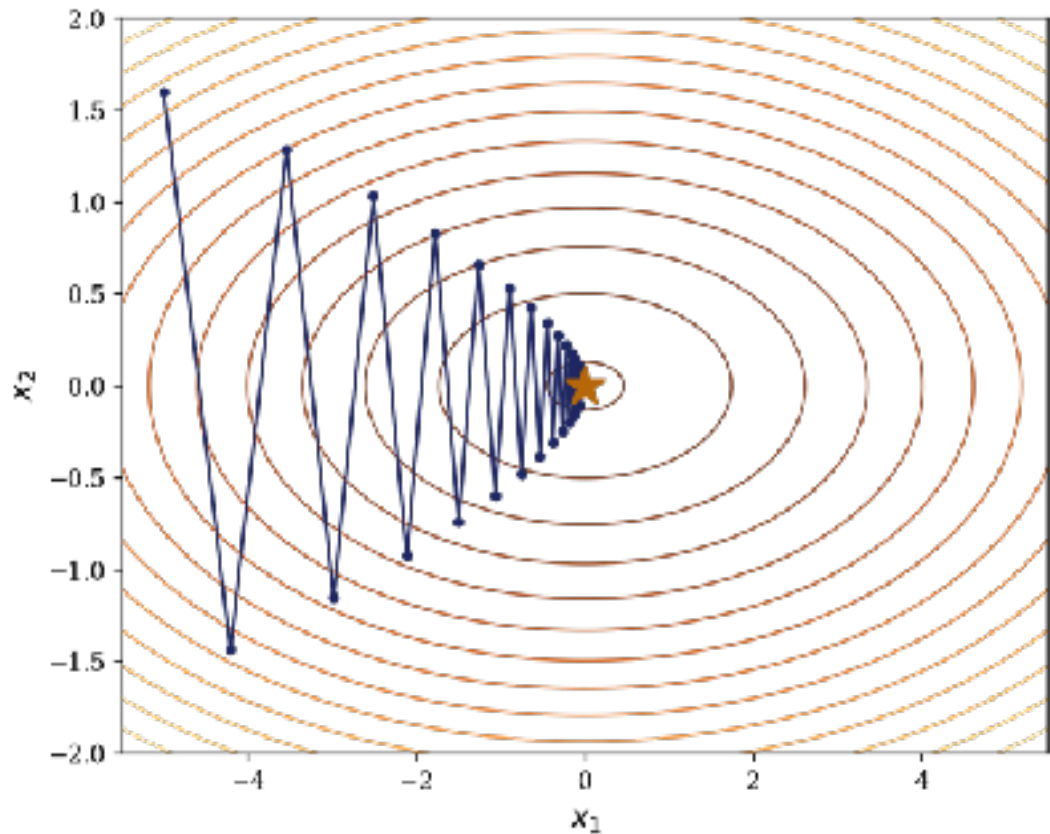
$$\mathbf{d}^{(k)} = -\frac{\mathbf{g}^{(k)}}{\|\mathbf{g}^{(k)}\|}$$

This is guaranteed to improve f for a sufficiently small step — provided the gradient isn't already zero. But choosing the maximally-decreasing step at every iteration has a surprising cost:

With exact line search, consecutive descent directions are always orthogonal: $\mathbf{d}^{(k+1)} \top \mathbf{d}^{(k)} = 0$. In narrow valleys, this forces many small zig-zagging steps.

5.1 GRADIENT DESCENT

Zig-Zagging in Narrow Valleys



Why it happens

In a narrow, elongated bowl, the gradient points almost perpendicular to the direction that actually leads to the minimum. Steepest descent repeatedly overshoots across the narrow axis while barely progressing along the long axis.

This is the same effect on the famous Rosenbrock “banana” function — curved, narrow valleys are exactly where gradient descent struggles most.

Fixes: momentum (this lecture), conjugate gradient (next), or second-order methods (Chapter 6).

5.2 CONJUGATE GRADIENT

Learning from the Quadratic Case

Conjugate gradient borrows inspiration from optimizing quadratic functions:

$$f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^\top \mathbf{A} \mathbf{x} + \mathbf{b}^\top \mathbf{x} + c$$

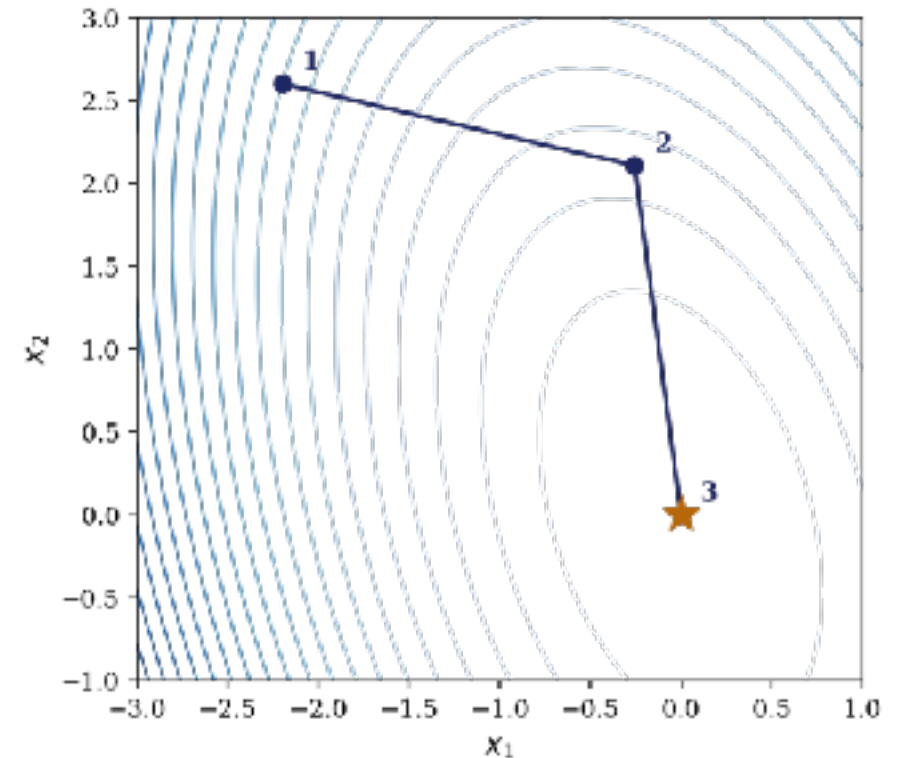
Its search directions are mutually conjugate with respect to A:

$$\mathbf{d}^{(i)\top} \mathbf{A} \mathbf{d}^{(j)} = 0 \text{ for all } i \neq j$$

Each new direction combines the current gradient with a contribution from the previous direction:

$$\mathbf{d}^{(k+1)} = -\mathbf{g}^{(k+1)} + \beta^{(k)} \mathbf{d}^{(k)}$$

For an n-dimensional quadratic, this converges in exactly n steps.



5.2 CONJUGATE GRADIENT

Choosing β for Non-Quadratic Functions

Smooth functions behave like quadratics near a local minimum, so conjugate gradient still works well — but we don't know the true A . Instead, two popular formulas estimate β from gradients alone:

Fletcher–Reeves

$$\beta^{(k)} = \frac{\mathbf{g}^{(k+1)\top} \mathbf{g}^{(k+1)}}{\mathbf{g}^{(k)\top} \mathbf{g}^{(k)}}$$

Polak–Ribière

$$\beta^{(k)} = \frac{\mathbf{g}^{(k+1)\top} (\mathbf{g}^{(k+1)} - \mathbf{g}^{(k)})}{\mathbf{g}^{(k)\top} \mathbf{g}^{(k)}}$$

Polak–Ribière can be made convergent with an automatic reset: $\beta \leftarrow \max(\beta, 0)$. When $\beta = 0$, we simply recover a plain gradient descent step.

Building Up Speed

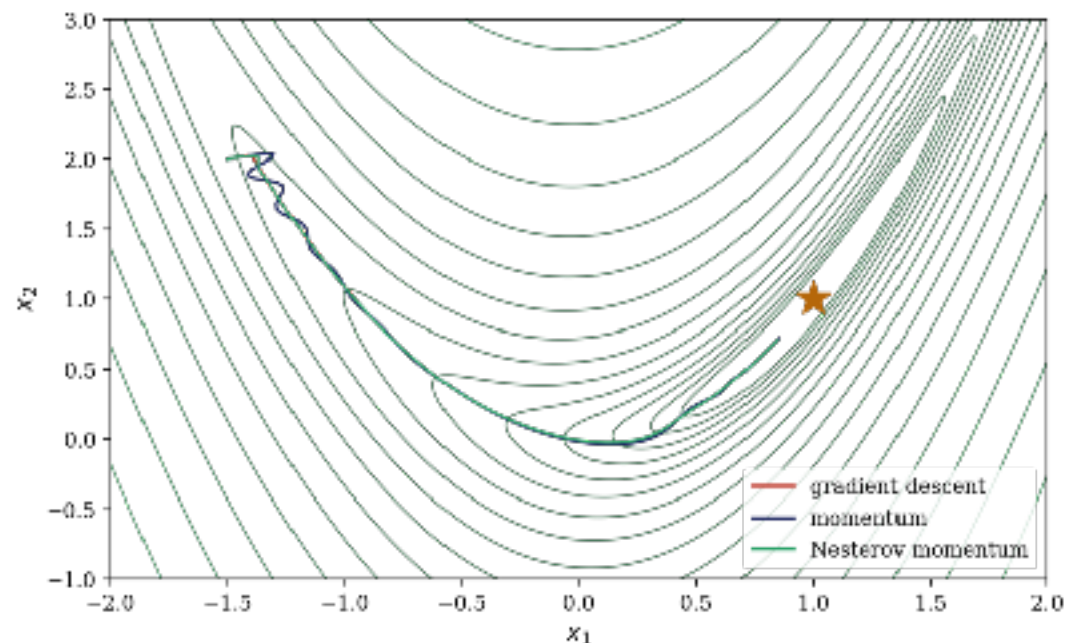
Momentum accumulates a velocity vector, letting progress build up across consecutive similar gradients — like a ball gathering speed rolling downhill:

$$\mathbf{v}^{(k+1)} = \beta \mathbf{v}^{(k)} - \alpha \mathbf{g}^{(k)} \quad \mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \mathbf{v}^{(k+1)}$$

Nesterov momentum evaluates the gradient at the projected future position, correcting for overshoot at the bottom of a valley:

$$\mathbf{v}^{(k+1)} = \beta \mathbf{v}^{(k)} - \alpha \nabla f(\mathbf{x}^{(k)} + \beta \mathbf{v}^{(k)})$$

Plain gradient descent (red) barely moves across the nearly-flat region — momentum (navy) and Nesterov momentum (green) carry speed through it and keep progressing toward the minimum.



One Learning Rate per Parameter

Momentum still applies the same learning rate to every component of x . Adaptive methods scale each component's rate individually.

Adagrad

$$x_i^{(k+1)} = x_i^{(k)} - \frac{\alpha}{\epsilon + \sqrt{s_i^{(k)}}} g_i^{(k)}$$

Divides by the accumulated sum of squared gradients — dulls parameters with consistently large gradients. Weakness: the sum only grows, so the rate can shrink to near zero.

RMSProp

$$\hat{s}^{(k+1)} = \gamma \hat{s}^{(k)} + (1 - \gamma)(g^{(k)} \odot g^{(k)})$$

Fixes Adagrad's decaying rate by using an exponentially decaying average of squared gradients instead of an ever-growing sum (decay γ typically ≈ 0.9). Adadelta is a closely related variant.

Both need an ϵ (e.g. 10^{-8}) in the denominator to avoid division by zero.

5.8 ADAM

Combining Momentum and Adaptive Rates

Adam (adaptive moment estimation) tracks decaying averages of both the gradient (momentum) and its square (adaptive rate), then bias-corrects them:

$$\mathbf{v}^{(k+1)} = \gamma_v \mathbf{v}^{(k)} + (1 - \gamma_v) \mathbf{g}^{(k)} \quad \mathbf{s}^{(k+1)} = \gamma_s \mathbf{s}^{(k)} + (1 - \gamma_s) (\mathbf{g}^{(k)} \odot \mathbf{g}^{(k)})$$

These estimate the true first and second moments of the gradient; correction removes their bias toward zero at early iterations. The update rule is then:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \frac{\alpha \hat{\mathbf{v}}^{(k+1)}}{\epsilon + \sqrt{\hat{\mathbf{s}}^{(k+1)}}}$$

Adam is widely used as a strong default in machine learning: it combines momentum's acceleration with per-parameter adaptive scaling, and is fairly robust to its hyperparameter choices.

First-Order Methods at a Glance

Method	Key idea	Learning rate	Good for...
Gradient descent	Step opposite the gradient	Fixed / decaying	Simple, well-conditioned problems
Conjugate gradient	Combine with previous direction	Line search	Quadratic-like, smooth problems
Momentum	Accumulate velocity	Fixed	Flat regions, mild ravines
Nesterov momentum	Look ahead before gradient	Fixed	Same as momentum, less overshoot
Adagrad / RMSProp	Per-parameter adaptive rate	Self-adjusting	Sparse or uneven gradients
Adam	Momentum + adaptive rate	Self-adjusting	Default choice in ML practice

All of these are first-order: they only ever look at the gradient. Chapter 6 brings in curvature (the Hessian) for faster, more targeted steps.

Part B — Key Takeaways

- 1 Gradient descent is simple but zig-zags in narrow, ill-conditioned valleys.
- 2 Conjugate gradient uses mutually conjugate directions, converging in n steps on quadratics.
- 3 Momentum and Nesterov momentum accumulate velocity to speed through flat or curved regions.
- 4 Adagrad, RMSProp, and Adam adapt the learning rate per-parameter using gradient history.
- 5 Adam — momentum plus adaptive, bias-corrected rates — is a strong general-purpose default.

PART C • CHAPTER 6

Second-Order Methods

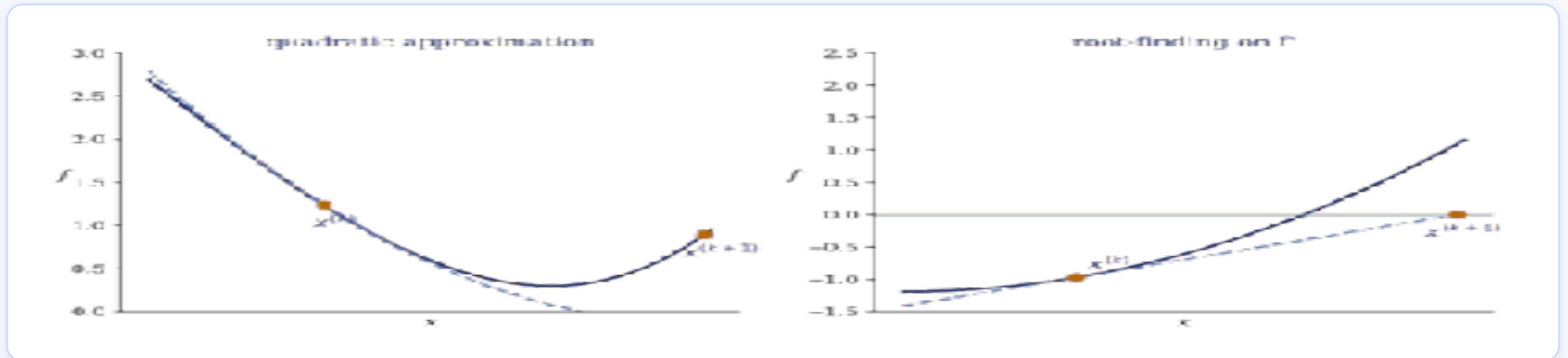
Newton's method, the secant method, and quasi-Newton methods

6.1 NEWTON'S METHOD

Using Curvature to Pick the Step

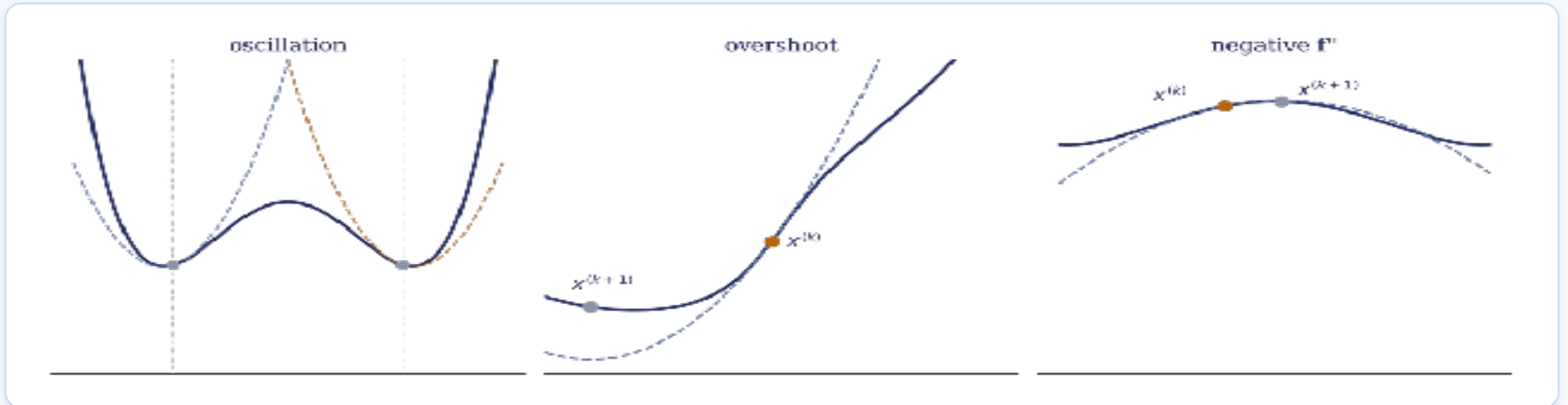
First-order information tells us which direction to move, but not how far. Second-order information — a quadratic (Taylor) approximation — lets us jump straight to where that approximation has zero gradient:

$$x^{(k+1)} = x^{(k)} - \frac{f'(x^{(k)})}{f''(x^{(k)})}$$



6.1 NEWTON'S METHOD

When Newton's Method Goes Wrong



Oscillation

The step bounces between two points forever without converging.

Overshoot

A too-flat local region produces a tiny curvature estimate and a huge, wild step.

Negative f''

Near a local maximum, the update moves toward the max instead of a min.

Newton's method converges quadratically near a well-behaved local minimum — but is not globally reliable. It's often combined with line search or a trust region for robustness.

6.1 NEWTON'S METHOD

The Multivariate Update

The gradient and Hessian generalize the univariate update directly:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - (\mathbf{H}^{(k)})^{-1} \mathbf{g}^{(k)}$$

Worked example: Booth's function

$$f(\mathbf{x}) = (x_1 + 2x_2 - 7)^2 + (2x_1 + x_2 - 5)^2$$

$$\nabla f(\mathbf{x}) = (10x_1 + 8x_2 - 34, \quad 8x_1 + 10x_2 - 38)$$

Starting from $\mathbf{x}(1) = [9, 8]$, a single Newton step lands exactly on $\mathbf{x}^* = [1, 3]$ — because Booth's function is quadratic, its Hessian is constant, and Newton's method solves quadratics in one step.

6.2 SECANT METHOD

Newton's Method Without the Second Derivative

Newton's method needs both f' and f'' . Often f' is known but f'' is not. The secant method estimates f'' from the last two gradient evaluations:

estimate f'' :

$$f''(x^{(k)}) \approx \frac{f'(x^{(k)}) - f'(x^{(k-1)})}{x^{(k)} - x^{(k-1)}}$$

substituted into Newton's update:

$$x^{(k+1)} \leftarrow x^{(k)} - \frac{x^{(k)} - x^{(k-1)}}{f'(x^{(k)}) - f'(x^{(k-1)})} f'(x^{(k)})$$

The trade-off: the secant method needs an additional initial point and suffers from the same failure modes as Newton's method — potentially made worse by approximating the second derivative.

6.3 QUASI-NEWTON METHODS

Approximating the Inverse Hessian

In multiple dimensions, quasi-Newton methods approximate the inverse Hessian $\mathbf{Q}$, updated incrementally from gradient differences:

$$\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} - \alpha^{(k)} \mathbf{Q}^{(k)} \mathbf{g}^{(k)}$$

Davidon–Fletcher–Powell (DFP)

$$\mathbf{Q} \leftarrow \mathbf{Q} - \frac{\mathbf{Q} \boldsymbol{\gamma} \boldsymbol{\gamma}^\top \mathbf{Q}}{\boldsymbol{\gamma}^\top \mathbf{Q} \boldsymbol{\gamma}} + \frac{\boldsymbol{\delta} \boldsymbol{\delta}^\top}{\boldsymbol{\delta}^\top \boldsymbol{\gamma}}$$

Broyden–Fletcher–Goldfarb–Shanno (BFGS)

$$\mathbf{Q} \leftarrow \mathbf{Q} - \frac{\boldsymbol{\delta} \boldsymbol{\gamma}^\top \mathbf{Q} + \mathbf{Q} \boldsymbol{\gamma} \boldsymbol{\delta}^\top}{\boldsymbol{\delta}^\top \boldsymbol{\gamma}} + \left(1 + \frac{\boldsymbol{\gamma}^\top \mathbf{Q} \boldsymbol{\gamma}}{\boldsymbol{\delta}^\top \boldsymbol{\gamma}}\right) \frac{\boldsymbol{\delta} \boldsymbol{\delta}^\top}{\boldsymbol{\delta}^\top \boldsymbol{\gamma}}$$

Both start with $\mathbf{Q} = \mathbf{I}$ and update it with each step, using only gradient information — no Hessian required.

L-BFGS & First- vs. Second-Order Trade-offs

Limited-memory BFGS (L-BFGS)

BFGS still stores a dense $n \times n$ matrix Q . For very large problems, L-BFGS instead stores only the last m gradient/step pairs and reconstructs the update on the fly — trading a little accuracy for large memory savings.

	First-order (Ch. 5)	Second-order (Ch. 6)
Information used	Gradient only	Gradient + Hessian (or approx.)
Cost per iteration	Low	Higher (Hessian) or moderate (quasi-Newton)
Convergence near minimum	Linear	Quadratic (Newton) / superlinear (quasi-Newton)
Storage	$O(n)$	$O(n^2)$ (Newton/BFGS) or $O(mn)$ (L-BFGS)

Part C — Key Takeaways

- 1 Newton's method uses a quadratic (Hessian-based) local model to jump toward a stationary point.
- 2 It converges quadratically near a minimum, but can oscillate, overshoot, or diverge elsewhere.
- 3 The secant method approximates f'' when only the gradient is available.
- 4 Quasi-Newton methods (DFP, BFGS) approximate the inverse Hessian using only gradient history.
- 5 L-BFGS makes quasi-Newton methods practical at scale by storing a short history instead of a full matrix.

From a Gradient to an Algorithm



Local descent

Choose a direction, choose a step (line search or trust region), and know when to stop.



First-order methods

Gradient descent, conjugate gradient, momentum, and adaptive rates like Adam.



Second-order methods

Newton's method and quasi-Newton methods use curvature for faster convergence.

Every algorithm in this lecture is an answer to the same two questions: which direction, and how far? The rest of the course explores answers that avoid gradients altogether.

Next Time: Beyond the Gradient

Chapters 4–6 assumed we could compute (or approximate) a gradient. Lecture 3 asks: what if we can't, or don't want to?

Ch. 7

Direct Methods

Search using only function evaluations — pattern search, Nelder-Mead, DIRECT.

Ch. 8

Stochastic Methods

Simulated annealing, cross-entropy method — randomness as a search tool.

Ch. 9

Population Methods

Genetic algorithms, particle swarm optimization — evolving many candidates together.

Questions?

Reading for next time: Algorithms for Optimization, Chapters 7–9